Date
Page

# XGBoost (extreme gradient boosting)

It is a highly optimized and scalable implementation of the gradient boosting algo designed for speed, performance, and accuracy.

Developed by chen and has become one of the most successful ML algo's in Kaggle competitions and real-world industry applications.

Key idea
Instead of building one complex model, XGBoost combines many weak decision trees sequentially, where each new tree tries to correct the errors made by the previous trees.

Why XGBoost became popular?
Traditional ML algorithms often struggle with:

- high-dimensional data
- missing values
- large datasets
- feature interactions
- Non-linear relationships

XGBoost addresses these challenges through:

- gradient boosting framework
- regularization
- parallel processing
- tree pruning
- missing value handling
- feature importance extraction
- distributed computing

3. evolution path

Decision tree

↓

Random forest

↓

gradient boosting machine (GBM)

↓

XGBoost

4. Refresher: what is a decision tree?

A decision tree splits data using conditions:

eg - Age > 30?

yes / \ No

Buy  Don't Buy

A single tree:

- easy to understand
- fast

But:
- often overfits
- can have high variance

5. What is boosting?

Boosting means:
'combining multiple weak learners to create a strong learner.'

eg: suppose we predict house prices.

Tree 1

Predict = 40,000

Actual Value: 45,000

error: 5,000

Tree 2 learns this error

Tree 3 learns remaining error

Final prediction = Tree 1 + Tree 2 + Tree 3...

6. Core idea of XGBoost

model prediction is: $\hat{y} = \sum_{k=1}^{K} f_k(x)$

where: $f_k$ = $k^{th}$ decision tree

$K$ = no. of decision trees

Final o/p:
Prediction =
tree 1 +
tree 2 +
tree 3 +
...
tree K

Each new tree minimizes previous errors.

4. How XGBoost works?

Step1: Initialize prediction
For regression:
Initial prediction = mean(y)

eg: house prices:
[100, 120, 130, 150]
mean = 125

Prediction starts with:
125 for all rows

Step 2: Calculate residuals.
Residual = Actual - Predicted

eg: 100 - 125 = -25
120 - 125 = -5
130 - 125 = 5
150 - 125 = 25

Step 3: Train first tree on residuals.
Tree tries to predict:
[-25, -5, 5, 25]

Step 4: Update prediction.
New prediction =
Old prediction +
learning rate * tree o/p

Step 5: Repeat
New residuals calculated.
New tree trained.
Continue until stopping criteria are met.

8. Why 'gradient' boosting?
The algorithm uses gradients from the loss func to determine the dirn of improvement.

Mathematically: $\text{Residual} = -\frac{\partial L}{\partial \hat{y}}$

where $L$ = loss func
$\hat{y}$ = prediction
The -ve gradient tells us how to improve the prediction.

9. Objective funcn of XGBoost

One of the most important interview questions;

XGBoost optimizes;

Obj = loss + regularization

expanded;

$$obj = \sum L(y_i, \hat{y}_i) + \sum \Omega(f_k)$$

where;

loss funcn

measures prediction error.

eg;

regression; $(y - \hat{y})^2$

classification; log loss

Regularization

Controls model complexity

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum w^2$$

T = no. of leaves  w = leaf weights

$\gamma$ = gamma  $\lambda$ = lambda

This prevents overfitting.

Q. XGBoost special features

A. Regularization
Unlike traditional GBM, XGBoost includes:

L1 Regularization

$$\alpha \Sigma |w|$$

benefits:
feature selection
sparsity

L2 Regularization

$$\lambda \Sigma w^2$$

benefits:
- smooth weights
- less overfitting

B. Pruning
Regular trees:
grow → stop

XGBoost
grow Deep
↓
Prune back

Only useful branches remain.

Result:

- faster
- better generalization

C. Missing value handling

Dataset:

| Age | Salary |
|---|---|
| 25 | 50K |
| NA | 60K |
| 30 | 70K |

XGBoost automatically learns:

If Age missing
→ go left

or

If Age missing
→ go right

No imputation is not needed in many cases.

D. Parallel Processing

Tree construction uses parallel computation.

Benefits:

- faster training
- scales to large datasets

E. Weighted Quantile Sketch
Efficient algo for finding split points.

Useful for:
- large datasets
- distributed systems

F. Out-of-core computing
Can handle datasets larger than RAM.
Data processed in chunks from disk.

11. Important hyperparameters

n_estimators (no. of trees)
more learning
more overfitting risk

Typical: 100-1000

learning_rate ($\eta$)

controls tree contribution.
New prediction = old + $\eta$ (Tree)
lower:
slower learning
better generalization
Typical: [0.01 - 0.3]

max_depth
maximum tree depth
eg: max_depth = 5

higher?
- more complexity
- higher overfitting

typical: 3-10

4. min_child_weight
minimum observations needed in a leaf.
higher value?
- conservative model
- less overfitting

5. gamma
minimum gain needed to split.
gamma = 0, 1
higher gamma?
- fewer splits
- simpler trees

6. subsample
fraction of rows used.
eg - subsample = 0.8
Benefits
- Reduces variance
- Helps generalization

classmate
Date
Page

7. colsample_bytree
fraction of columns used
eg - colsample_bytree = 0.8
Acts like Random Forest feature sampling

8. lambda
L2 regularization
lambda = 1

9. alpha
L1 regularization
alpha = 0.5

R. Bias-Variance Perspective
Underfitting
1. low depth
2. few trees
3. high bias

Overfitting
1. many trees
2. high depth
3. low bias
4. high variance

13. feature importance
XGBoost provides feature importance metrics.

- Gain
improvement in accuracy due to feature
most commonly used

Weight
no. of times feature used

Cover
no. of observations affected

eg: income = 45%
age = 30%
gender = 15%
city = 10%

14. Classification with XGBoost

Tasks:
binary classification
multi-class

eg: spam detection
fraud "
disease prediction
customer churn

Binary classification o/p:
0.85

Meaning:
85% prob. of class 1

Prediction:
if prob > 0.5:
class 1
else:
class 0

15. Regression with XGBoost
eg: house prices
sales forecasting
demand "
stock trend modeling

evaluation metrics:
MAE, MSE, RMSE, $R^2$

17. limitations of XGBoost
Not very interpretable
compared to a single decision tree

Hyperparameter tuning required
performance depends heavily on tuning

Training time: can be slow with thousands of trees

memory usage
Largers datasets may consume more memory

18. XGBoost vs Random forest :

| Feature | XGBoost | Random forest |
|---|---|---|
| learning style | sequential | parallel |
| performance | usually higher | good |
| speed | slower training | faster |
| overfitting control | strong | moderate |
| hyperparameter tuning | more | less |
| accuracy | usually better | good |

19. XGBoost vs lightGBM vs catBoost

| Feature | XGBoost | LightGBM | CatBoost |
|---|---|---|---|
| Speed | fast | very fast | fast |
| Accuracy | high | high | high |
| categorical supp. | limited | limited | excellent |
| memory usage | medium | low | medium |
| ease of use | moderate | moderate | easy |

Rules of thumb;
XGBoost → general purpose, robust choice
lightGBM → massive datasets
CatBoost → many categorical variables

## Python Implementation

```
pip install xgboost
```

### basic classification example

```
from xgboost import xGBClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris= load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.2,
    random_state = 42
)

model = XGBClassifier(
        n_estimator = 100,
        learning_rate = 0.1,
        max_depth = 3,
        random_state = 42
)
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

## Feature Importance

```
import matplotlib.pyplot as plt
from xgboost import plot_importance

plot_importance(model)
plt.show()
```

o/p eg:

| | |
|---|---|
| Income | 40% |
| House age | 20% |
| Latitude | 15% |
| Longitude | 10% |
| Others | 15% |

## Early stopping

if validation score stops improving
→ stop training
→ avoid overfitting

```
model.fit(
    X_train,
    y_train,
    eval_set = [(X_test, y_test)],
    verbose = True
)
```